Does ARCHITECTURE.md Change How a Coding Agent Works?

This report compares two conditions on Python-Markdown 3.11.2: code only (A) versus the same code plus a two-page architecture document (B). Three tasks and two trials each yield twelve agent runs scored on acceptance tests, regressions, design fit, time, and tool use.

1. Hypotheses

RQ1 (outcome). I expected condition B to show a higher complete rate on the cross-component task (T3), because the document names the Extension contract, entry points in pyproject.toml, and the rule that core must not import extensions. For T1 and T2 I expected both conditions to succeed often, since the bugs and features are local to one extension file.

RQ2 (quality). I expected B to produce fewer regressions and higher design-fit scores, especially on T3, where violating C2 or C3 is easy if the agent patches core.py instead of adding tasklist.py plus an entry point.

RQ3 (efficiency). I expected B to need fewer tool calls and less time on T2 and T3 because the overview diagram and ADR-2 shorten the search for where to register processors. I did not expect a large effect on T1 once the agent opens toc.py.

2. Setup

Repository. Python-Markdown, https://github.com/Python-Markdown/markdown.git, tag 3.11.2, commit b41e0aaf1181f1a8cb200cf5a815aad7c0020d23. Ninety-one documentation paths were removed (experiment/removed_files.txt); the upstream README was replaced by a short install-and-test note. Baseline before the injected T1 bug: 1077 passed, 0 failed, 52 skipped (1129 tests run). experiment/cond-A is the prepared tree with the bug; cond-B adds ARCHITECTURE.md only.

Tasks. T1 (bug): empty heading ids in the toc extension must use the existing _1, _2 numbering. T2 (local feature): ??? admonitions as collapsible <details> blocks in admonition.py. T3 (cross-component): new built-in tasklist extension loadable as tasklist or markdown.extensions.tasklist. Each task has a 5-10 line spec sent to the agent, four hidden acceptance tests, and three design-fit checks (see experiment/tasks/Tn.md).

Agent and settings. Each run used a fresh Cursor generalPurpose subagent with model Grok 4.7 (inherit), provider-default sampling, 30-minute limit, and sandbox C:\Users\anilk\autose_sandboxes\<run_id> copied from cond-A or cond-B. The prompt is the fenced spec plus "When you are done, make sure the existing tests still pass." Condition B adds "Read ARCHITECTURE.md before you start." No hints. Part 4 replayed each diff.patch with pip install -e ".[testing]".

3. Architecture document

Production. The first draft was written in Cursor with Grok 4.7 while preparing the repository: an eight-row component table, mermaid pipeline diagram, three ADRs, and five constraints with file citations. There was no separate prompt file. A second pass on 5 November 2026 checked ten factual claims against experiment/cond-A; two were corrected (Extension import also in __init__.py; serializers.py does not import util.py). ADR-1 and ADR-2 are Verified from commits 6ee07d2 and cd73243; ADR-3 mechanism is Verified from HtmlStash docstring with Inferred alternatives. Total authoring time was roughly three hours including the claim check. Only cond-B receives the file.

Ten claims checked against the code

#	Claim	File:line	OK?
1	Default output format is xhtml	core.py:130	Yes
2	Tree processors inline 20, prettify 10, unescape 0	treeprocessors.py:42-44	Yes
3	Block processor priorities 100 down to 10	blockprocessors.py:45-55	Yes
4	Higher Registry priority runs first	util.py:377-379	Yes
5	Extension load: entry point, module:Class, makeExtension	core.py:208-214	Yes
6	toc tree processor priority 5	extensions/toc.py:492	Yes
7	attr_list priority 8 before toc	extensions/attr_list.py:215	Yes
8	NormalizeWhitespace strips STX/ETX	preprocessors.py:71	Yes
9	Only core imports Extension	__init__.py:46 too	Fixed in C3
10	util imported everywhere	serializers.py: no util	Fixed in overview

4. Results

Table 1 lists all twelve runs. Every run reached Complete on the four hidden acceptance tests for its task, with zero regressions on the baseline unittest set (975 named tests matched from baseline_tests.txt). Design fit varied by task, not by condition.

Table 1. Per-run scores

run_id	task	cond	trial	outcome	acc	reg	fit	time	tools	tok
T1-A-1	T1	A	1	Complete	4/4	0	2	6.2	39	-
T1-B-1	T1	B	1	Complete	4/4	0	2	5.1	34	-
T1-A-2	T1	A	2	Complete	4/4	0	2	3.5	24	-
T1-B-2	T1	B	2	Complete	4/4	0	2	4.7	25	-
T2-A-1	T2	A	1	Complete	4/4	0	3	6.8	36	-
T2-B-1	T2	B	1	Complete	4/4	0	3	5.9	37	-
T2-A-2	T2	A	2	Complete	4/4	0	3	7.0	40	-
T2-B-2	T2	B	2	Complete	4/4	0	3	9.0	62	-
T3-A-1	T3	A	1	Complete	4/4	0	1	16.7	87	-
T3-B-1	T3	B	1	Complete	4/4	0	1	12.4	69	-
T3-A-2	T3	A	2	Complete	4/4	0	1	15.9	92	-
T3-B-2	T3	B	2	Complete	4/4	0	1	16.8	82	-

Table 2. Summary by condition (mean over six runs)

Condition	Complete rate	Mean regressions	Mean design fit	Mean time (min)	Mean tool calls
A (no doc)	100%	0.0	2.00	9.3	53
B (+ ARCHITECTURE.md)	100%	0.0	2.00	9.0	52

By task, mean design fit was 2.0 on T1 (missing regression test in most diffs), 3.0 on T2, and 1.0 on T3 in both conditions. Mean time on T3 was about 16 min without the document and 15 min with it-within run-to-run noise.

5. Analysis

RQ1. All twelve runs Complete on acceptance tests. The architecture document did not change the binary outcome in this sample: agents fixed T1, shipped T2, and passed T3 functionally in every trial. That does not mean the tasks were too easy overall-T3 still required a new module-but it does mean RQ1 cannot separate A from B on success rate here.

RQ2. Regressions were zero everywhere. Design fit differed by task: T2 runs scored 3/3 in both conditions; T1 scored 2/3 because agents fixed toc.py but usually omitted a dedicated regression test; T3 scored 1/3 in both conditions because every agent edited markdown/core.py to load the short name tasklist instead of relying on the pyproject.toml entry point alone-contrary to ADR-2 and constraint C3 even when B read ARCHITECTURE.md. The document did not prevent that shortcut.

RQ3. Mean tool calls were 53 (A) versus 52 (B); mean time 9.4 min versus 9.0 min. B was slightly faster on T1 and T3 trials but slower on T2-B-2 (62 tools, 9.0 min). With only two trials per cell, those differences are descriptive, not evidence that the document saves time.

Qualitative runs

Strong run T2-B-2 (condition B). The agent stated it would read the architecture notes first, opened ARCHITECTURE.md, then edited only admonition.py-extending AdmonitionProcessor for ??? markers and treating <details> like for indented continuations. The document aligned with the intended module boundary.

Weak run T3-A-1 (condition A). The agent created tasklist.py and the entry point, but when extensions=['tasklist'] failed before reinstall it patched build_extension in core.py. Acceptance tests passed after install; design fit was 1/3. The log shows the failure probe immediately followed by a core.py StrReplace. Condition B runs on T3 behaved the same on design fit, so reading ADRs did not stop the core shortcut in this study.

6. Limitations

Only two trials per task and condition; all outcomes were Complete, so there is no variance to compare on RQ1. I authored the tasks, the hidden tests, and the architecture document, which may match Grok's training priors. The agent is non-deterministic: T2-B-2 used 62 tool calls while T2-A-1 used 36 despite the same spec. Scoring used replayed diffs on Windows with one Python version. Condition C (overview-only ARCHITECTURE.md for T3) was not run. Token/cost columns are empty because subagent logs did not report usage.

7. Conclusion

In these twelve runs, ARCHITECTURE.md did not measurably improve acceptance outcomes or design fit on the hardest task. It may have helped agents choose the right extension file on T2 when they actually read it, but it did not stop core.py edits on T3. Correct behaviour and architecturally correct change decouple when the agent can patch the loader. Fair comparison and scored diffs made that visible; the evidence supports "little difference on outcomes, limited effect on structure," not "documentation always helps."